

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 12 – SEARCHING



NAMA : Bambang Darmawan Saputra

NIM : 109082530003

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Pencarian data (searching) merupakan salah satu operasi paling mendasar dan penting dalam algoritma dan pemrograman yang bertujuan untuk menemukan keberadaan suatu nilai data spesifik di dalam sekumpulan data (array atau collection). Berbeda dengan pencarian nilai ekstrem (seperti mencari nilai maksimum atau minimum) yang hasilnya selalu pasti ditemukan di dalam kumpulan data tersebut, operasi pencarian data spesifik memiliki kemungkinan bahwa data yang dicari tidak terdaftar atau tidak ditemukan. Hasil dari algoritma pencarian ini umumnya tidak hanya mengonfirmasi status keberadaan data lewat nilai boolean (true/false), melainkan juga mengembalikan posisi indeks atau lokasi memori dari data tersebut agar dapat diproses lebih lanjut. Jika data tidak ditemukan, sistem biasanya diatur untuk mengembalikan nilai penanda khusus seperti angka -1 atau bilangan di luar batas indeks larik yang valid.

Salah satu metode pencarian yang paling sederhana adalah Sequential Search atau pencarian berurutan, yang bekerja dengan mengecek setiap elemen larik satu per satu dari elemen pertama hingga elemen terakhir. Ciri khas utama dari algoritma ini adalah proses pencarian akan langsung dihentikan segera setelah elemen yang dicari berhasil ditemukan, meskipun masih terdapat elemen-elemen selanjutnya di dalam larik yang belum diperiksa secara manual. Karena tidak bergantung pada posisi susunan elemen, Sequential Search dapat digunakan secara fleksibel baik pada sekumpulan data yang acak maupun data yang sudah terurut. Namun, karena kompleksitasnya yang berbanding lurus dengan jumlah data ($O(N)$), metode ini menjadi kurang efisien ketika diterapkan pada sistem penyimpanan data berskala besar.

Untuk mengatasi kelemahan efisiensi tersebut, terdapat algoritma Binary Search yang memanfaatkan kondisi keterurutan data (baik terurut membesar/ascending maupun mengecil/descending) guna memangkas waktu pencarian secara drastis. Cara kerja algoritma ini adalah dengan langsung memeriksa elemen posisi tengah (median) dari rentang data yang ada, lalu membandingkannya dengan nilai yang dicari. Jika nilai tengah tersebut lebih kecil atau lebih besar dari target, algoritma akan membuang setengah rentang data yang tidak relevan dan menggeser fokus pencarian ke setengah sisa rentang data lainnya. Karakteristik pemangkasan rentang menjadi setengah di setiap langkahnya membuat Binary Search sangat efisien ($O(\log N)$). Meski begitu, kelemahan utama dari Binary Search adalah sifat ketergantungan mutlak pada urutan data; apabila data belum terurut atau urutannya terbalik dari logika perbandingan algoritma, pencarian dipastikan gagal sehingga data harus diurutkan terlebih dahulu atau dialihkan menggunakan Sequential Search.

B. Guided

1. [guided1]

```
package main

import "fmt"

type arrData [5]string //tipe data alias

func seqSearch(arr arrData, binatangCari string) int {
    var found int = -1
    for i := 0; i < len(arr); i++ {
        if arr[i] == binatangCari {
            found = i
            break
        }
    }
    return found
}

func main() {
    var arrBinatang arrData //array tipe string

    for i := 0; i < len(arrBinatang); i++ {
        fmt.Printf("Masukkan data binatang indeks ke-%d : ", i)
        fmt.Scan(&arrBinatang[i])
    }
    fmt.Println()

    var binatangCari string
    fmt.Print("Masukkan nama binatang yang mau dicari : ")
    fmt.Scan(&binatangCari)

    var idxCari int
    idxCari = seqSearch(arrBinatang, binatangCari)

    if idxCari > -1 {
        fmt.Printf("Data %s ditemukan pada indeks ke-%d!",
binatangCari, idxCari)
    } else if idxCari == -1 {
        fmt.Printf("Data %s tidak ditemukan!", binatangCari)
    }
}
```

Output :

```
PS D:\github\109082530003_Bambang-Darmawan-Saputra-1> go run "d:\github\109082530003_Bambang-Darmawan-Saputra-1\guided.go"
Masukkan data binatang indeks ke-0 : cacing
Masukkan data binatang indeks ke-1 : anjing
Masukkan data binatang indeks ke-2 : naga
Masukkan data binatang indeks ke-3 : kucing
Masukkan data binatang indeks ke-4 : burung

Masukkan nama binatang yang mau dicari : naga
Data naga ditemukan pada indeks ke-2!
```

Penjelasan :

Program ini menyimpan lima nama binatang dalam array arrBinatang, lalu mencari nama yang dimasukkan pengguna dengan fungsi seqSearch; jika ditemukan, dicetak indeksinya, kalau tidak, muncul pesan tidak ditemukan.

2. [guided2]

```
5package main

import "fmt"

// fungsi binary search
func binarySearch(data []int, x int) int {

    // batas kiri array
    kiri := 0

    // batas kanan array
    kanan := len(data) - 1

    // perulangan selama kiri <= kanan
    for kiri <= kanan {

        // mencari index tengah
        tengah := (kiri + kanan) / 2

        // jika data ditemukan
        if data[tengah] == x {
            return tengah
        }

        // jika x lebih besar, cari ke kanan
        } else if data[tengah] < x {
            kiri = tengah + 1
        }

        // jika x lebih kecil, cari ke kiri
        } else {
            kanan = tengah - 1
        }
    }
}
```

```

    }

    // jika data tidak ditemukan
    return -1
}

func main() {

    var n int
    var cari int

    // input jumlah data
    fmt.Print("Jumlah data: ")
    fmt.Scan(&n)

    // membuat array sesuai jumlah data
    data := make([]int, n)

    // input isi array
    fmt.Println("Masukkan data terurut:")
    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    // input angka yang dicari
    fmt.Print("Angka yang dicari: ")
    fmt.Scan(&cari)

    // memanggil fungsi binary search
    hasil := binarySearch(data, cari)

    // output hasil pencarian
    if hasil != -1 {
        fmt.Println("Data ditemukan di index:", hasil)
    } else {
        fmt.Println("Data tidak ditemukan")
    }
}

```

Output :

```

PS D:\github\109082530003_Bambang-Darmawan-Saputra-1> go run "d:\github\109082530003_Bambang-Darmawan-Saputra-1\guided2.go"
Jumlah data: 5
Masukkan data terurut:
1
2
3
4
5
Angka yang dicari: 1
Data ditemukan di index: 0

```

Penjelasan :

Program ini membaca data terurut dan angka yang dicari, lalu menggunakan binarySearch untuk menemukan indeks angka atau mengembalikan -1 jika tidak ada.

3. [guided3]

```

package main

import "fmt"

type mahasiswa struct {
    nama string
    nim   int
    IPK   float64
}

type arrMahasiswa [5]mahasiswa

func binSearchStruct(arrData arrMahasiswa, nimDicari int) int {
    var found int = -1
    var med int
    var kr int = 0
    var kn int = len(arrData) - 1
    for kr <= kn && found == -1 {
        med = (kr + kn) / 2
        if nimDicari < arrData[med].nim {
            kn = med - 1
        } else if nimDicari > arrData[med].nim {
            kr = med + 1
        } else {
            found = med
        }
    }
    return found
}

```

```

func seqSearchStruct(arrData arrMahasiswa, nimDicari int) int {
    var found int = -1
    for i := 0; i < len(arrData); i++ {
        if arrData[i].nim == nimDicari {
            found = i
            break
        }
    }
    return found
}

func main() {
    var mahasiswa06 arrMahasiswa
    var nimDicari int

    fmt.Println("--- MASUKKAN DATA NIM MAHASISWA SECARA TERURUT ---")
    for i := 0; i < len(mahasiswa06); i++ {
        fmt.Printf("=== Masukkan data mahasiswa 06 pada indeks ke-%d\n", i)
        fmt.Print("Masukkan nama : ")
        fmt.Scan(&mahasiswa06[i].nama)
        fmt.Print("Masukkan NIM : ")
        fmt.Scan(&mahasiswa06[i].nim)
        fmt.Print("Masukkan IPK : ")
        fmt.Scan(&mahasiswa06[i].IPK)
        fmt.Println("-----")
    }
    fmt.Println()

    fmt.Print("Masukkan NIM mahasiswa yang ingin dicari : ")
    fmt.Scan(&nimDicari)
    fmt.Println()

    var idxHasilCariSeqSearch int
    idxHasilCariSeqSearch = seqSearchStruct(mahasiswa06, nimDicari)

    fmt.Println("== HASIL SEQUENTIAL SEARCH ==")
    if idxHasilCariSeqSearch > -1 {
        fmt.Printf("Data NIM mahasiswa %d ditemukan pada array di indeks ke-%d!\n", nimDicari, idxHasilCariSeqSearch)
        fmt.Println("Berikut Detail Datanya : ")
        fmt.Printf("Nama : %s\n", mahasiswa06[idxHasilCariSeqSearch].nama)
    }
}

```

```

        fmt.Printf("NIM : %d\n",
mahasiswa06[idxHasilCariSeqSearch].nim)
        fmt.Printf("IPK : %.2f\n",
mahasiswa06[idxHasilCariSeqSearch].IPK)
    } else if idxHasilCariSeqSearch == -1 {
        fmt.Printf("Data NIM mahasiswa %d tidak ditemukan pada
array!", nimDicari)
    }
    fmt.Println()

    var idxHasilCariBinSearch int
    idxHasilCariBinSearch = binSearchStruct(mahasiswa06, nimDicari)

    fmt.Println("== HASIL BINARY SEARCH ==")
    if idxHasilCariBinSearch > -1 {
        fmt.Printf("Data NIM mahasiswa %d ditemukan pada array di
indeks ke-%d!\n", nimDicari, idxHasilCariBinSearch)
        fmt.Println("Berikut Detail Datanya : ")
        fmt.Printf("Nama : %s\n",
mahasiswa06[idxHasilCariBinSearch].nama)
        fmt.Printf("NIM : %d\n",
mahasiswa06[idxHasilCariBinSearch].nim)
        fmt.Printf("IPK : %.2f\n",
mahasiswa06[idxHasilCariBinSearch].IPK)
    } else if idxHasilCariBinSearch == -1 {
        fmt.Printf("Data NIM mahasiswa %d tidak ditemukan pada
array!", nimDicari)
    }
}

```

Output :


```

PS D:\github\109082530003_Bambang-Darmawan-Saputra-1> go run "d:\github\109082530003_Bambang-Darmawan-Saputra-1\guided3.go"
--- MASUKKAN DATA NIM MAHASISWA SECARA TERURUT ---
=== Masukkan data mahasiswa 06 pada indeks ke-0 ===
Masukkan nama : putra
Masukkan NIM : 10
Masukkan IPK : 3
-----
=== Masukkan data mahasiswa 06 pada indeks ke-1 ===
Masukkan nama : sap
Masukkan NIM : 20
Masukkan IPK : 4
-----
=== Masukkan data mahasiswa 06 pada indeks ke-2 ===
Masukkan nama : ma
Masukkan NIM : 09
Masukkan IPK : 2
-----
=== Masukkan data mahasiswa 06 pada indeks ke-3 ===
Masukkan nama : re
Masukkan NIM : 54
Masukkan IPK : 3
-----
=== Masukkan data mahasiswa 06 pada indeks ke-4 ===
Masukkan nama : fa
Masukkan NIM : 2
Masukkan IPK : 4
-----

Masukkan NIM mahasiswa yang ingin dicari : 10

== HASIL SEQUENTIAL SEARCH ==
Data NIM mahasiswa 10 ditemukan pada array di indeks ke-0!
Berikut Detail Datanya :
Nama : putra
NIM : 10
IPK : 3.00

== HASIL BINARY SEARCH ==
Data NIM mahasiswa 10 tidak ditemukan pada array!

```

Penjelasan :

Program ini membaca lima data mahasiswa berurutan, lalu mencari nim yang diminta dengan sequential search dan binary search, dan menampilkan indeks plus detail mahasiswa jika ditemukan atau pesan tidak ditemukan.

C. Unguided

1. Soal Unguided 1

Pada pemilihan ketua RT yang baru saja berlangsung, terdapat 20 calon ketua yang bertanding memperebutkan suara warga. Perhitungan suara dapat segera dilakukan karena warga cukup mengisi formulir dengan nomor dari calon ketua RT yang dipilihnya. Seperti biasa, selalu ada pengisian yang tidak tepat atau dengan nomor pilihan di luar yang tersedia, sehingga data juga harus divalidasi. Tugas Anda untuk membuat program mencari siapa yang memenangkan pemilihan ketua RT.

Buatlah program **pilkart** yang akan membaca, memvalidasi, dan menghitung suara yang diberikan dalam pemilihan ketua RT tersebut.

Masukan hanya satu baris data saja, berisi bilangan bulat valid yang kadang tersisipi dengan data tidak valid. Data valid adalah integer dengan nilai di antara 1 s.d. 20 (inklusif). Data berakhir jika ditemukan sebuah bilangan dengan nilai 0.

Keluaran dimulai dengan baris berisi jumlah data suara yang terbaca, diikuti baris yang berisi berapa banyak suara yang valid. Kemudian sejumlah baris yang mencetak data para calon apa saja yang mendapatkan suara.

Jawaban:

```
package main

import "fmt"

func main() {
    var suara, suaraMasuk, suaraSah int
    var hitungSuara [21]int // Indeks 1 s.d 20 untuk calon ketua RT

    for {
        fmt.Scan(&suara)
        if suara == 0 {
            break
        }
        suaraMasuk++
        if suara >= 1 && suara <= 20 {
            suaraSah++
            hitungSuara[suara]++
        }
    }

    fmt.Printf("Suara masuk: %d\n", suaraMasuk)
    fmt.Printf("Suara sah: %d\n", suaraSah)
    for i := 1; i <= 20; i++ {
```

```

        if hitungSuara[i] > 0 {
            fmt.Printf("%d:%d\n", i, hitungSuara[i])
        }
    }
}

```

Output :

```

PS D:\github\109082530003_Bambang-Darmawan-Saputra-1> go run "d:\github\109082530003_Bambang-Darmawan-Saputra-1\unguided.go"
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
1:1
2:1
3:2
7:1
18:1
19:2

```

Penjelasan :

Program ini menghitung total suara masuk, menyaring suara sah (kandidat nomor 1–20), lalu menampilkan jumlah suara masing-masing kandidat secara berurutan. Proses pembacaan data langsung berhenti saat mendeteksi angka 0.

2. Soal Unguided 2

Berdasarkan program sebelumnya, buat program **pilkart** yang mencari siapa pemenang pemilihan ketua RT. Sekaligus juga ditentukan bahwa wakil ketua RT adalah calon yang mendapatkan suara terbanyak kedua. Jika beberapa calon mendapatkan suara terbanyak yang

sama, ketua terpilih adalah dengan nomor peserta yang paling kecil dan wakilnya dengan nomor peserta terkecil berikutnya.

Masukan hanya satu baris data saja, berisi bilangan bulat valid yang kadang tersisipi dengan data tidak valid. Data valid adalah bilangan bulat dengan nilai di antara 1 s.d. 20 (inklusif). Data berakhir jika ditemukan sebuah bilangan dengan nilai 0.

Keluaran dimulai dengan baris berisi jumlah data suara yang terbaca, diikuti baris yang berisi berapa banyak suara yang valid. Kemudian tercetak calon nomor berapa saja yang menjadi pasangan ketua RT dan wakil ketua RT yang baru.

Jawaban :

```
package main

import "fmt"

func main() {
    var suara, suaraMasuk, suaraSah int
    var hitungSuara [21]int

    for {
        fmt.Scan(&suara)
        if suara == 0 {
            break
        }
        suaraMasuk++
        if suara >= 1 && suara <= 20 {
            suaraSah++
            hitungSuara[suara]++
        }
    }
}
```

```

    }
}

fmt.Printf("Suara masuk: %d\n", suaraMasuk)
fmt.Printf("Suara sah: %d\n", suaraSah)

ketua := -1
wakil := -1

// Mencari Ketua RT (Suara terbanyak, nomor terkecil jika seri)
for i := 1; i <= 20; i++ {
    if hitungSuara[i] > 0 {
        if ketua == -1 || hitungSuara[i] > hitungSuara[ketua] {
            ketua = i
        }
    }
}

// Mencari Wakil RT (Suara terbanyak kedua, nomor terkecil jika
seri)
for i := 1; i <= 20; i++ {
    if hitungSuara[i] > 0 && i != ketua {
        if wakil == -1 || hitungSuara[i] > hitungSuara[wakil] {
            wakil = i
        }
    }
}

fmt.Printf("Ketua RT: %d\n", ketua)
fmt.Printf("Wakil ketua: %d\n", wakil)
}

```

Output :

```

PS D:\github\109082530003_Bambang-Darmawan-Saputra-1> go run "d:\github\109082530003_Bambang-Darmawan
-Saputra-1\unguided2.go"
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
Ketua RT: 3
Wakil ketua: 19

```

Penjelasan :

Program ini menentukan Ketua RT berdasarkan suara terbanyak, dan Wakil RT dari suara terbanyak kedua. Jika ada kandidat dengan jumlah suara yang sama (seri), pemenang ditentukan dari nomor kandidat yang paling kecil.

3. Soal Unguided 3

Diberikan n data integer positif dalam keadaan terurut membesar dan sebuah integer lain k , apakah bilangan k tersebut ada dalam daftar bilangan yang diberikan? Jika ya, berikan indeksnya, jika tidak sebutkan "TIDAK ADA".

Masukan terdiri dari dua baris. Baris pertama berisi dua buah integer positif, yaitu n dan k . n menyatakan banyaknya data, dimana $1 < n \leq 1000000$. k adalah bilangan yang ingin dicari.

Baris kedua berisi n buah data integer positif yang sudah terurut membesar.

Keluaran terdiri dari satu baris saja, yaitu sebuah bilangan yang menyatakan posisi data yang dicari (k) dalam kumpulan data yang diberikan. Posisi data dihitung dimulai dari angka 0. Atau memberikan keluaran "TIDAK ADA" jika data k tersebut tidak ditemukan dalam kumpulan.

Program yang dibangun harus menggunakan subprogram dengan mengikuti kerangka yang sudah diberikan berikut ini.

Jawaban :

```
package main

import "fmt"

const NMAX = 1000000
var data [NMAX]int

func main() {
    var n, k int
    fmt.Scan(&n, &k)

    isiArray(n)

    idx := posisi(n, k)
    if idx != -1 {
        fmt.Println(idx)
    } else {
        fmt.Println("TIDAK ADA")
    }
}

func isiArray(n int) {
```

```

    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }
}

func posisi(n, k int) int {
    kr := 0
    kn := n - 1
    found := -1

    for kr <= kn && found == -1 {
        med := (kr + kn) / 2
        if data[med] == k {
            found = med
        } else if data[med] < k {
            kr = med + 1
        } else {
            kn = med - 1
        }
    }

    return found
}

```

Output :

```

PS D:\github\109082530003_Bambang-Darmawan-Saputra-1> go run "d:\github\109082530003_Bambang-Darmawan-Saputra-1\unguided3.go"
12 535
1 3 8 16 32 123 323 323 534 543 823 999
TIDAK ADA

```

Penjelasan :

Program ini mencari indeks dari sebuah angka k di dalam kumpulan data yang sudah terurut membesar. Menggunakan metode Binary Search (bagi dua rentang), program akan mencetak posisi indeksinya (dimulai dari 0) jika ketemu, atau mencetak "TIDAK ADA" jika gagal.

D. Kesimpulan

Berdasarkan hasil praktikum mengenai algoritma pencarian, dapat disimpulkan bahwa pemilihan metode pencarian data harus disesuaikan dengan kondisi struktur data yang dihadapi dan kebutuhan efisiensi program. Sequential Search sangat cocok digunakan untuk data berukuran kecil atau data acak yang tidak terurut karena implementasinya yang sederhana dan tidak memerlukan proses manipulasi urutan di awal. Sebaliknya, untuk menangani volume data yang sangat besar hingga mencapai jutaan elemen seperti pada kasus struktur data struct mahasiswa maupun data numerik besar, Binary Search menjadi pilihan mutlak karena kecepatan pencariannya yang superior. Namun,

pemanfaatan Binary Search menuntut syarat wajib berupa data yang harus sudah terurut secara konsisten berdasarkan field atau kunci pencarian yang digunakan agar pembagian rentang tengah tidak menghasilkan keluaran yang keliru